



EKS Session

Summary 27-05-2023

- AWS provides **Container Insights** as a service in CloudWatch to monitor containers in EKS. Container Insights collects metrics and logs from your containerized applications.
- The agent program that runs on top of EKS is called the **CloudWatch agent**. The CloudWatch agent collects metrics, logs, and traces from your containerized applications and microservices.
- If you only want to push metrics to Container Insights, then you can use the **Fluentd program**. Fluentd is an open-source data collector that can be used to collect data from a variety of sources, including containers. Fluentd can then send this data to a variety of destinations, including Container Insights.
- The CloudWatch & Fluentd Agent should be installed in each worker node in EKS for container monitoring. The Agent is a program that can be launched using a pod.
- If the number of worker nodes increases, it is not possible to manually launch the agent on each node. A **DaemonSet** is a Kubernetes resource that ensures that a pod is running on every node in a cluster. This makes it easy to deploy the CloudWatch Agent on all worker nodes in an EKS cluster.
- A DaemonSet is a Kubernetes resource that ensures that a specific pod runs on each node in a cluster. By deploying the CloudWatch Agent as a DaemonSet, you can automate the process of installing and running the agent on new worker nodes as they are added to the cluster.

- Using a **Helm chart**, launch a multi-tier application. We require a helm command for this. Therefore, install the Helm version 3 command that you downloaded from the internet.
- Check the version of Helm.

```
C:\Users\Vimal Daga\Downloads\helm-v3.11.3-windows-amd64\windows-amd64>helm.exe version
version.BuildInfo{Version:"v3.11.3", GitCommit:"323249351482b3bbfc9f5004f65d400aa70f9ae7", GitTreeState:"clean", GoVersion:"go1.20.3"}
```

- Add the Bitnami repository to Helm

```
C:\Users\Vimal Daga\Downloads\helm-v3.11.3-windows-amd64\windows-amd64>helm.exe repo add bitnami https://charts.bitnami.com/bitnami
"bitnami" has been added to your repositories
```

- Verify that the Bitnami repository has been added.

```
C:\Users\Vimal Daga\Downloads\helm-v3.11.3-windows-amd64\windows-amd64>helm.exe repo list
NAME      URL
eks       https://aws.github.io/eks-charts
bitnami   https://charts.bitnami.com/bitnami
```

- Search the WordPress chart in the Bitnami repo.

```
C:\Users\Vimal Daga\Downloads\helm-v3.11.3-windows-amd64\windows-amd64>helm.exe search repo wordpress
NAME                CHART VERSION  APP VERSION  DESCRIPTION
bitnami/wordpress  16.1.9         6.2.2        WordPress is the world's most popular blogging ...
bitnami/wordpress-intel 2.1.31        6.1.1        DEPRECATED WordPress for Intel is the most popu...
```

- Create a new namespace in K8S.

```
C:\Users\Vimal Daga>kubectl get pods -n lwproject
No resources found in lwproject namespace.
```

- Launch WordPress using the above chart in the created namespace.

```
C:\Users\Vimal Daga\Downloads\helm-v3.11.3-windows-amd64\windows-amd64>helm.exe install mylwpapp bitnami/wordpress -n lwproject
```

- Get the pods, service, and deployment.

```
C:\Users\Vimal Daga>kubectl get pods -n lwproject
NAME                                READY   STATUS    RESTARTS   AGE
mylwwpapp-mariadb-0                0/1     Pending   0           26s
mylwwpapp-wordpress-578c8f9687-795kr 0/1     Pending   0           26s

C:\Users\Vimal Daga>kubectl get svc -n lwproject
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      AGE
mylwwpapp-mariadb                   ClusterIP            10.100.2.193    <none>            35s
mylwwpapp-wordpress                 LoadBalancer        10.100.164.62   a6670c2d293064a76a2f7054609d7a1a-2010812888.ap-south-1.elb.amazonaws.com 80:30391/TCP,443:30940/TCP 35s

C:\Users\Vimal Daga>kubectl get deployment -n lwproject
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
mylwwpapp-wordpress                 0/1     1             0           56s
```

- You can modify the existing IAM role associated with the worker nodes to enable an EC2 instance (worker node) in Amazon EKS to send logs and metrics to CloudWatch.
- The CloudWatch & Fluentd Agent program installed on the EC2 instances will be able to utilize these permissions to send the desired data to CloudWatch automatically.
- In this cluster, find the IAM role that was created.

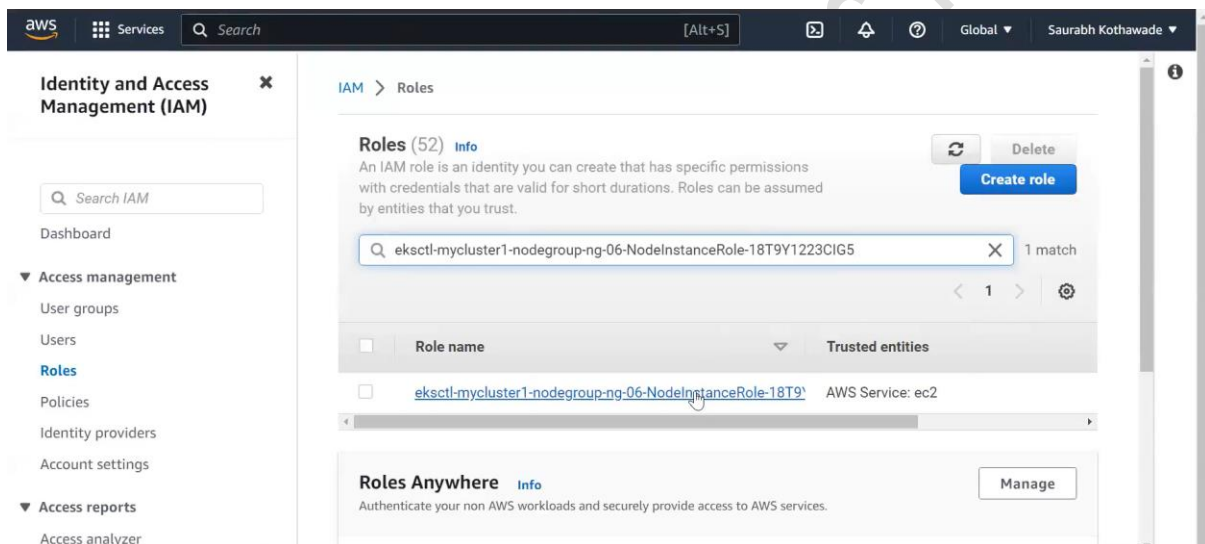
```
C:\Users\Vimal Daga>eksctl get cluster
NAME      REGION    EKSCluster  CREATED
mycluster1 ap-south-1 True

C:\Users\Vimal Daga>eksctl get nodegroup --cluster mycluster1
CLUSTER  NODEGROUP  STATUS  CREATED  MIN SIZE  MAX SIZE  DESIRED CAPACITY
TY        INST  TYPE  IMAGE ID  ASG NAME
mycluster1 ng-06013a54 ACTIVE  2023-05-27T05:46:59Z  2  2  2
2          m5.large AL2_x86_64 eks-ng-06013a54-dac42ddf-70ea-fbeb-65a4-e7800a11c68c managed

C:\Users\Vimal Daga>eksctl get nodegroup --cluster mycluster1 -o json _
```

```
C:\Users\Vimal Daga>eksctl get nodegroup --cluster mycluster1 -o json
[
  {
    "StackName": "eksctl-mycluster1-nodegroup-ng-06013a54",
    "Cluster": "mycluster1",
    "Name": "ng-06013a54",
    "Status": "ACTIVE",
    "MaxSize": 2,
    "MinSize": 2,
    "DesiredCapacity": 2,
    "InstanceType": "m5.large",
    "ImageID": "AL2_x86_64",
    "CreationTime": "2023-05-27T05:46:59.754Z",
    "NodeInstanceRoleARN": "arn:aws:iam::881559863141:role/eksctl-mycluster1-nodegroup-ng-06-NodeInstanceRole-18T9Y1223CIG5",
    "AutoScalingGroupName": "eks-ng-06013a54-dac42ddf-70ea-fbeb-65a4-e7800a11c68c",
    "Version": "1.24",
    "Type": "managed"
  }
]
```

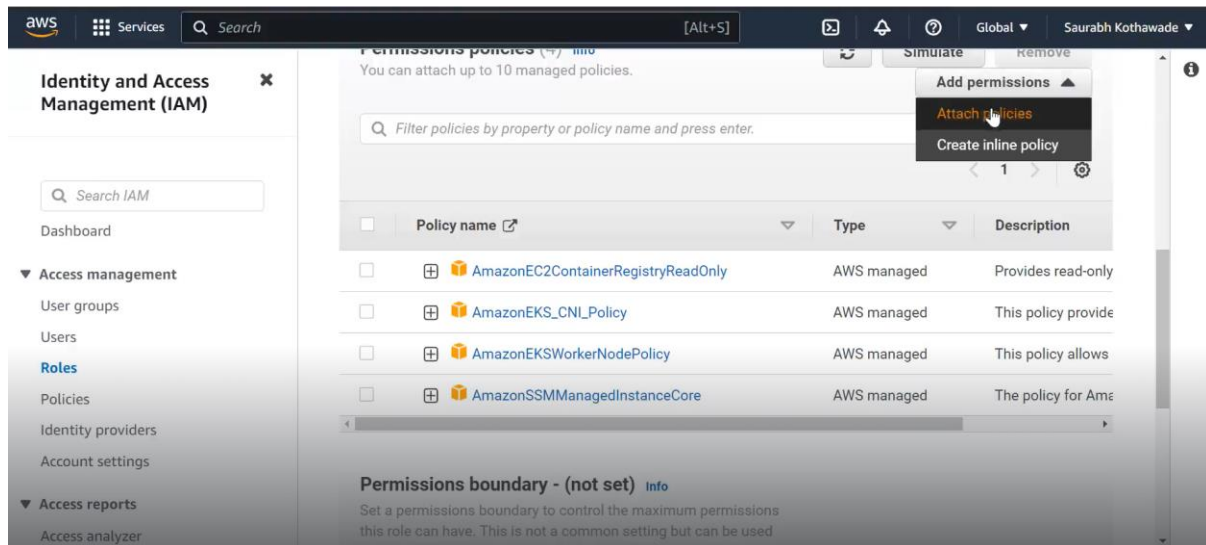
- Go to the IAM AWS console now and search for the IAM role that we found in the cluster information.



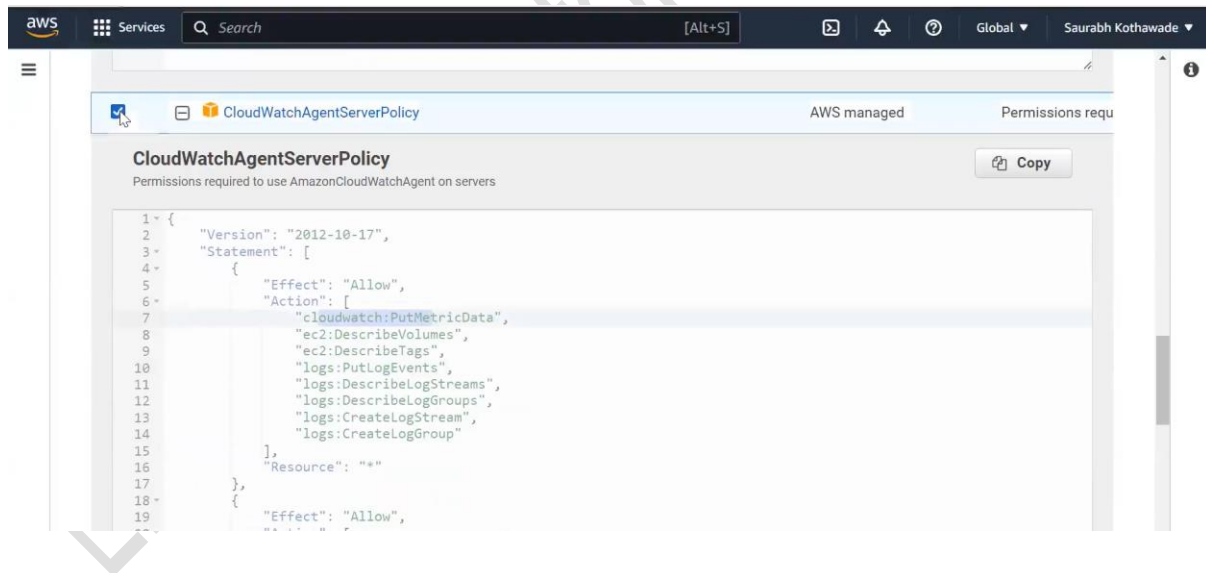
- There, we can see that it only has these permissions. don't have the ability to insert metrics into Cloudwatch.

☐	Policy name ↗	Type	Description
☐	AmazonEC2ContainerRegistryReadOnly	AWS managed	Provides read-only
☐	AmazonEKS_CNI_Policy	AWS managed	This policy provide
☐	AmazonEKSWorkerNodePolicy	AWS managed	This policy allows
☐	AmazonSSMManagedInstanceCore	AWS managed	The policy for Ame

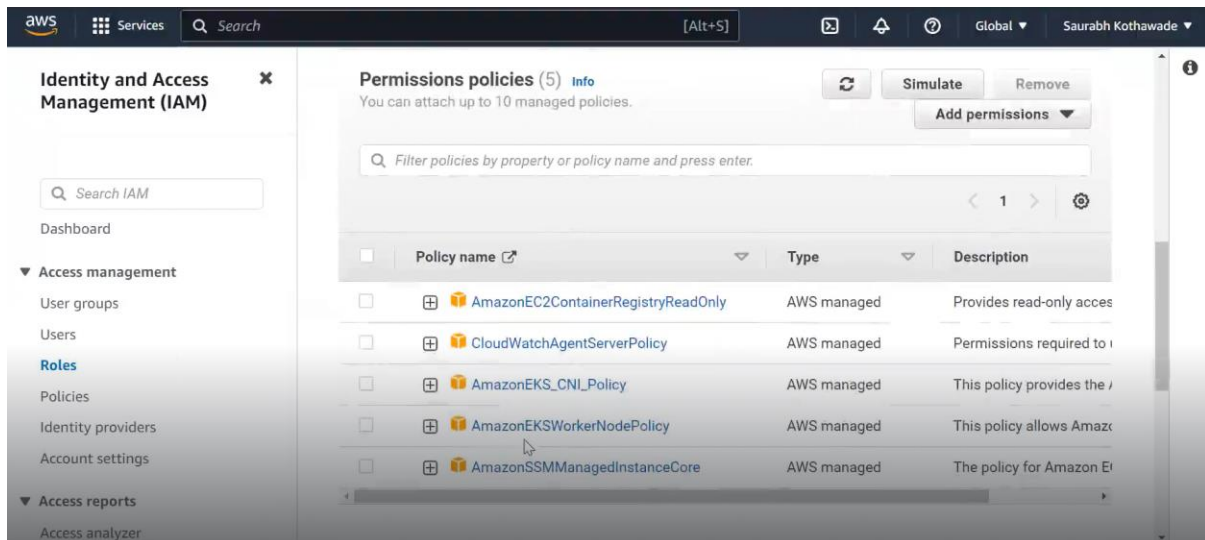
- Add CloudWatch permissions: In the Permissions tab of the IAM role, click on the "Attach policies" button.



- In the search box, type "**CloudWatchAgentServerPolicy**" and select the "CloudWatchAgentServerPolicy" policy. This policy provides the necessary permissions for the EC2 instance to send logs and metrics to CloudWatch.



- Click on "Attach permissions" to add the selected policy to the IAM role.
- Adding the "CloudWatchAgentServerPolicy" to the IAM role associated with the EKS worker nodes grants the EC2 instances the required permissions to send logs and metrics data to CloudWatch.



- Download the YAML code for deploying the CloudWatch agent and Fluentd agent on an Amazon EKS cluster.

```
$ curl https://raw.githubusercontent.com/aws-samples/amazon-cloudwatch-container-insights/latest/k8s-deployment-manifest-templates/deployment-mode/daemonset/container-insights-monitoring/quickstart/cwagent-fluentd-quickstart.yaml | sed "s/{{cluster_name}}/mycluster1/s/{{region_name}}/ap-south-1/" > EnableContainerInsights.yaml
```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload	Upload	Total	Spent	Left
100	16515	100	16515	0	0	7148	0
0:00:02	0:00:02	--:--:--	7164				

```
# curl https://raw.githubusercontent.com/aws-samples/amazon-cloudwatch-container-insights/latest/k8s-deployment-manifest-templates/deployment-mode/daemonset/container-insights-monitoring/quickstart/cwagent-fluentd-quickstart.yaml | sed "s/{{cluster_name}}/mycluster1/s/{{region_name}}/ap-south-1/" > EnableContainerInsights.yaml
```

- The provided YAML configuration is for deploying the CloudWatch agent and Fluentd agent on an Amazon EKS cluster. These agents will enable container insights and log forwarding to CloudWatch Logs.
- This will create the necessary resources in the amazon-cloudwatch namespace, including the CloudWatch agent daemonset and Fluentd agent deployment.


```

C:\Users\Vimal Daga\Downloads>kubectl apply -f EnableContainerInsights.yaml.txt
namespace/amazon-cloudwatch created
serviceaccount/cloudwatch-agent created
clusterrole.rbac.authorization.k8s.io/cloudwatch-agent-role created
clusterrolebinding.rbac.authorization.k8s.io/cloudwatch-agent-role-binding created
configmap/cwagentconfig created
daemonset.apps/cloudwatch-agent created
configmap/cluster-info created
serviceaccount/fluentd created
clusterrole.rbac.authorization.k8s.io/fluentd-role created
clusterrolebinding.rbac.authorization.k8s.io/fluentd-role-binding created
configmap/fluentd-config created
daemonset.apps/fluentd-cloudwatch created

```

- After applying the configuration, the CloudWatch agent and Fluentd agent will be deployed to your EKS cluster, and container insights will be enabled. The logs from the containers will be forwarded to CloudWatch Logs for monitoring and analysis.
- Get the namespace, daemonsets, and pods.

```

C:\Users\Vimal Daga\Downloads>kubectl get ns
NAME                STATUS  AGE
amazon-cloudwatch   Active  22s
default              Active  83m
kube-node-lease     Active  83m
kube-public         Active  83m
kube-system         Active  83m
lwproject           Active  66m

```

```

C:\Users\Vimal Daga\Downloads>
C:\Users\Vimal Daga\Downloads>kubectl get daemonsets -n amazon-cloudwatch
NAME                DESIRED  CURRENT  READY  UP-TO-DATE  AVAILABLE  NODE SELECTOR  AGE
cloudwatch-agent    2        2        2      2           2          <none>         43s
fluentd-cloudwatch  2        2        2      2           2          <none>         42s

C:\Users\Vimal Daga\Downloads>kubectl get pods -n amazon-cloudwatch
NAME                                READY  STATUS   RESTARTS  AGE
cloudwatch-agent-mwgmb             1/1    Running  0          64s
cloudwatch-agent-qqsww             1/1    Running  0          64s
fluentd-cloudwatch-mqhsc           1/1    Running  0          63s
fluentd-cloudwatch-qc69v           1/1    Running  0          63s

```

- You can see that each node has launched a fluentd agent pod and a cloudwatch agent pod.

```

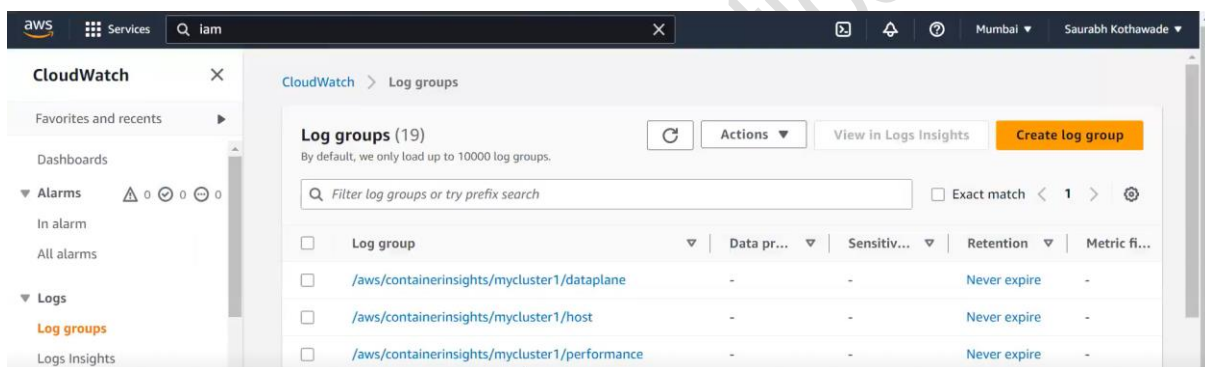
C:\Users\Vimal Daga\Downloads>kubectl get pods -o wide -n amazon-cloudwatch
NAME                                READY    STATUS    RESTARTS   AGE    IP                NODE
cloudwatch-agent-mwgmmb             1/1     Running   0           89s    192.168.49.219    ip-192-168-47-94.ap-south-1.
compute.internal                    <none>   <none>    <none>     <none>
cloudwatch-agent-qqsww              1/1     Running   0           89s    192.168.77.252    ip-192-168-73-177.ap-south-1.
compute.internal                    <none>   <none>    <none>     <none>
fluentd-cloudwatch-mqhscc           1/1     Running   0           88s    192.168.60.91     ip-192-168-47-94.ap-south-1.
compute.internal                    <none>   <none>    <none>     <none>
fluentd-cloudwatch-qc69v            1/1     Running   0           88s    192.168.67.180    ip-192-168-73-177.ap-south-1.
compute.internal                    <none>   <none>    <none>     <none>

C:\Users\Vimal Daga\Downloads>kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
ip-192-168-47-94.ap-south-1.compute.internal Ready    <none>   78m    v1.24.13-eks-0a21954
ip-192-168-73-177.ap-south-1.compute.internal Ready    <none>   78m    v1.24.13-eks-0a21954

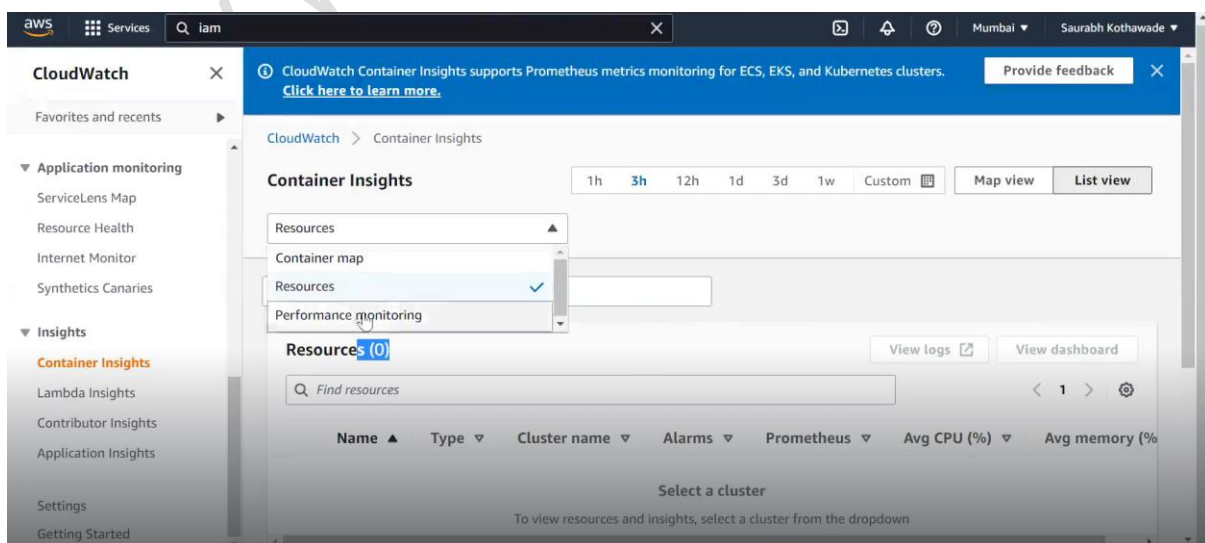
```

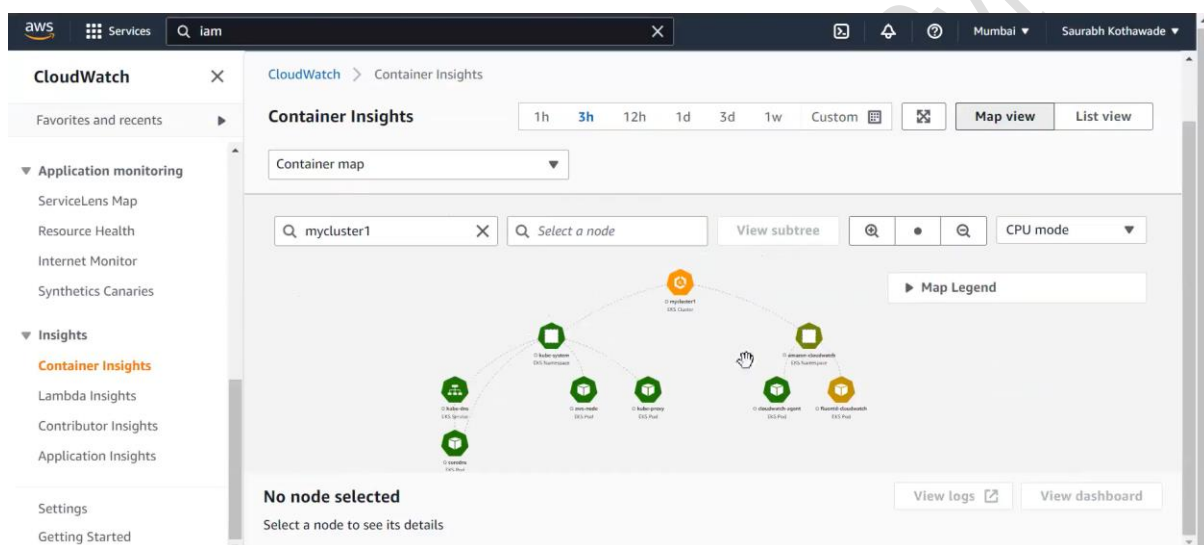
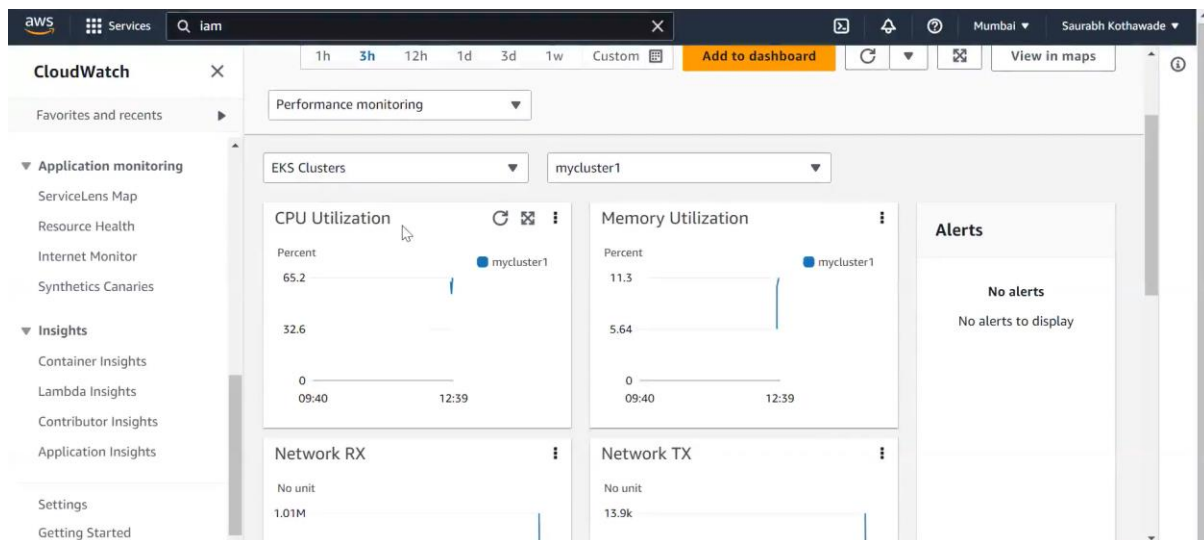
- **To verify the CloudWatch and Fluentd setup for Container Insights:**

1. Open the CloudWatch console
2. In the navigation pane, choose Log groups.
3. In the list of log groups in the Region, you should see the following:

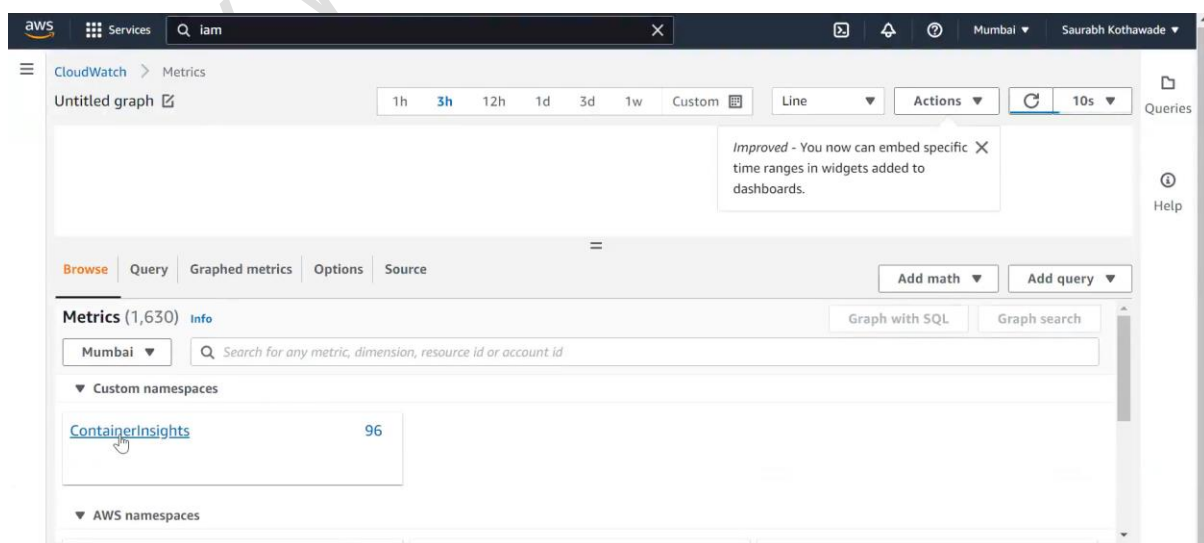


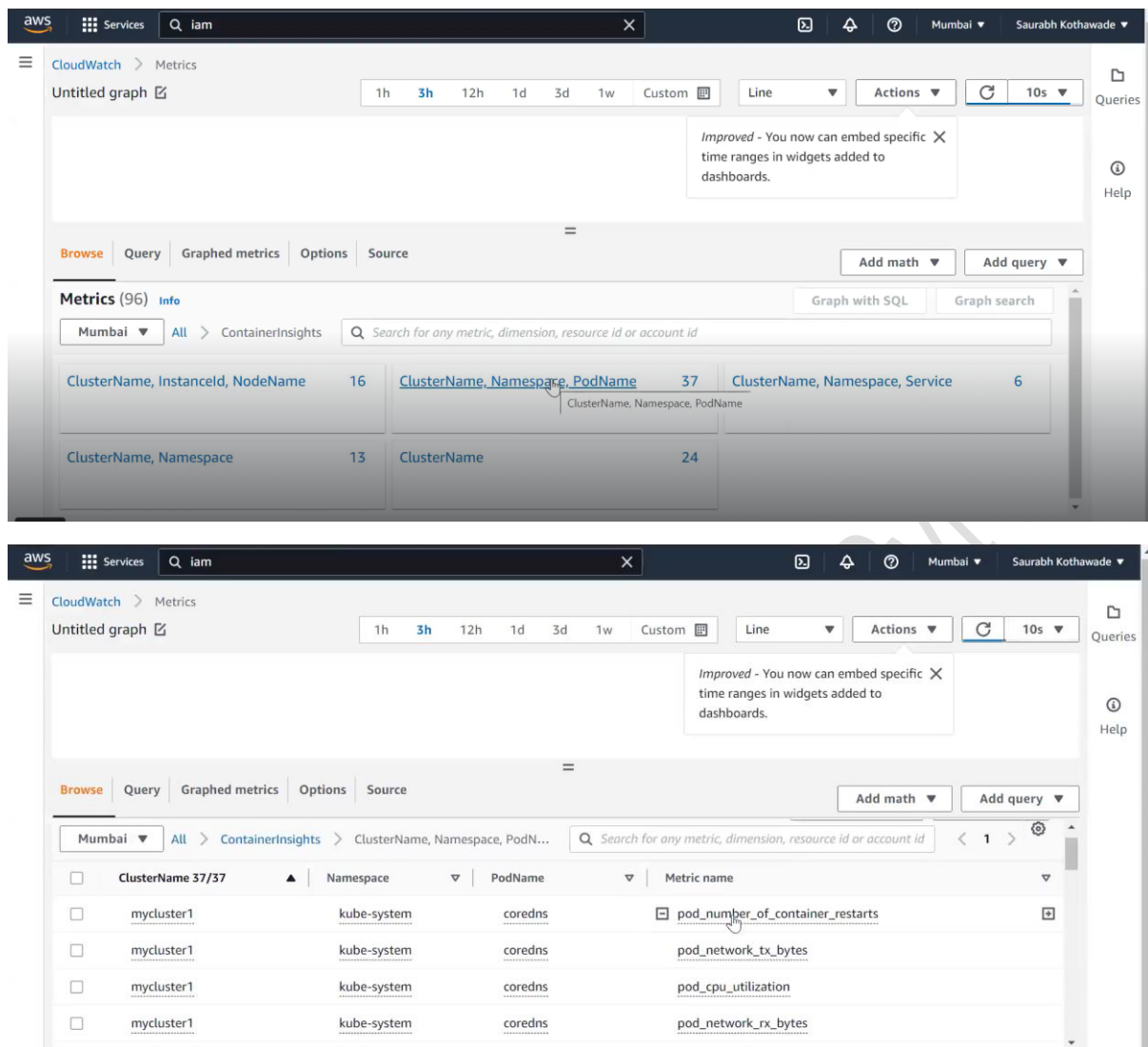
- **View Container Insights:** In the navigation pane, choose Container Insights. Then choose Performance monitoring.





- Metric will provide the data, which you can see in the Cloudwatch dashboard.

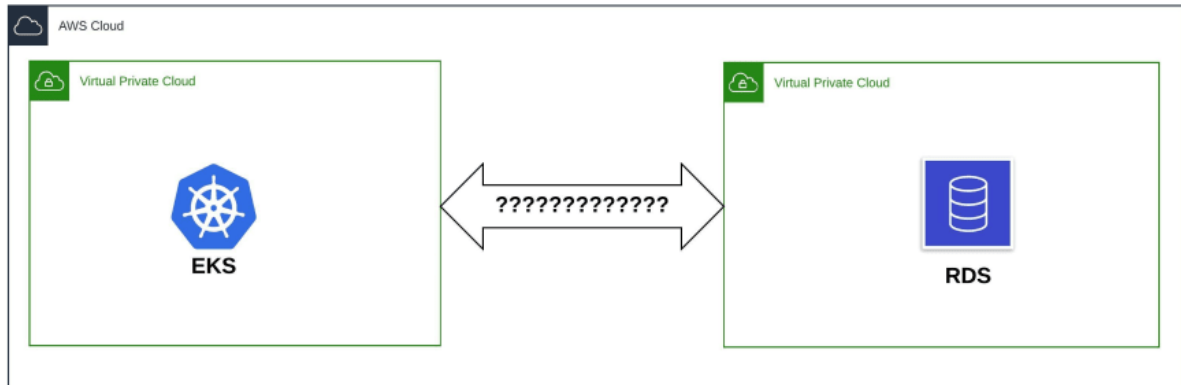




● Accessing Amazon RDS From AWS EKS

1. Accessing Amazon RDS (Relational Database Service) from AWS EKS (Elastic Kubernetes Service) involves establishing a connection between the Kubernetes cluster and the RDS instance.
2. EKS cluster runs your application pods, and the Amazon RDS instance hosts your database. The pods within the cluster need to communicate with the RDS instance to perform database operations.
3. The EKS cluster and the RDS instance should be located within the same VPC (Virtual Private Cloud) or have a peered connection to allow communication.

4. In your EKS cluster, create a Kubernetes Service object that acts as a stable endpoint for your application to connect to the RDS instance. The service should be of type **ExternalName** and point to the RDS endpoint.



- Amazon Elastic Kubernetes Service (EKS) has two primary mechanisms for autoscaling your worker nodes: **Auto Scaling Groups (ASG)** and **Karpenter**.

1. Auto Scaling Groups (ASG):

Auto Scaling Groups are a feature provided by Amazon EC2 that allows you to automatically adjust the number of EC2 instances in a group based on specified policies.

With ASG, you can configure scaling policies based on metrics such as CPU utilization, memory utilization, or custom metrics.

2. Karpenter:

Karpenter is an open-source project designed for cluster autoscaling in Kubernetes. A Kubernetes controller manages the scaling of worker nodes based on the resource demands of pods in the cluster.

Karpenter works in conjunction with the Horizontal Pod Autoscaler (HPA), which scales the number of pods based on metrics such as CPU utilization or custom metrics.

- Unlike ASG, which focuses on vertical scaling of EC2 instances, Karpenter enables horizontal scaling by adding or removing worker nodes. When the HPA determines that more pods are needed, Karpenter provisions additional worker nodes to accommodate the increased workload. Conversely, Karpenter removes unnecessary worker nodes to save resources when the workload decreases.
- Karpenter is a more advanced autoscaling solution that provides fine-grained control over pod placement and scheduling. It optimizes resource utilization by dynamically adjusting the number of worker nodes based on pod requirements.

Linux World Informatics Pvt Ltd